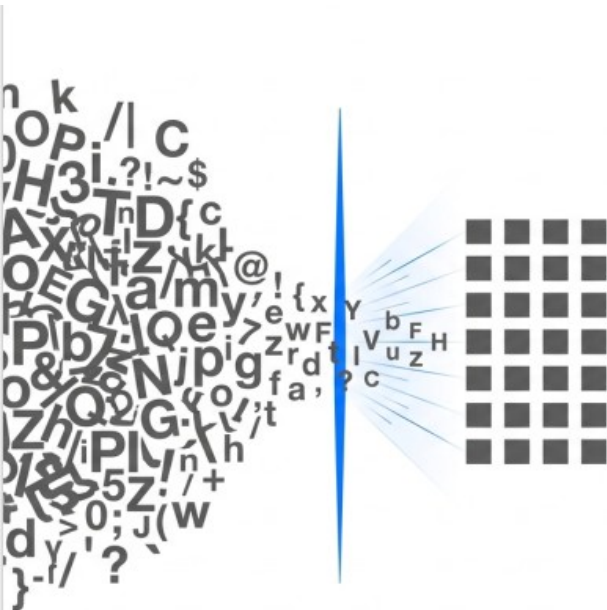


Unidad 2:

Procesamiento de Textos

de datos no estructurados a significado
indexable



Objetivos

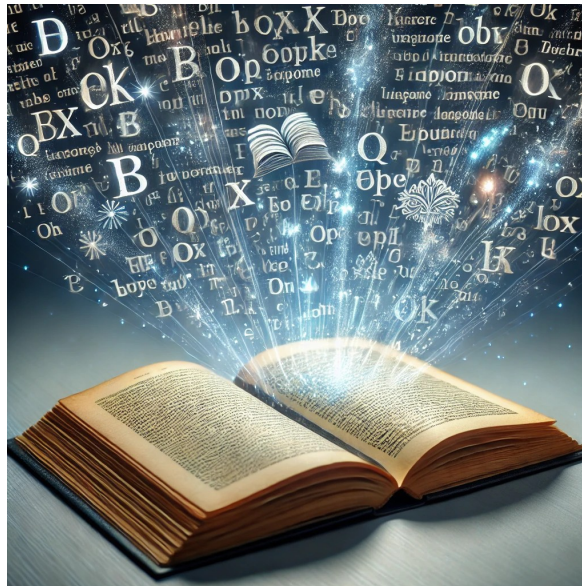
Entender cuales son los procesos básicos para el procesamiento de textos

Resumen

- Motivación.
- Pasos del procesamiento documental.
- Midiendo la importancia de un término

Motivación

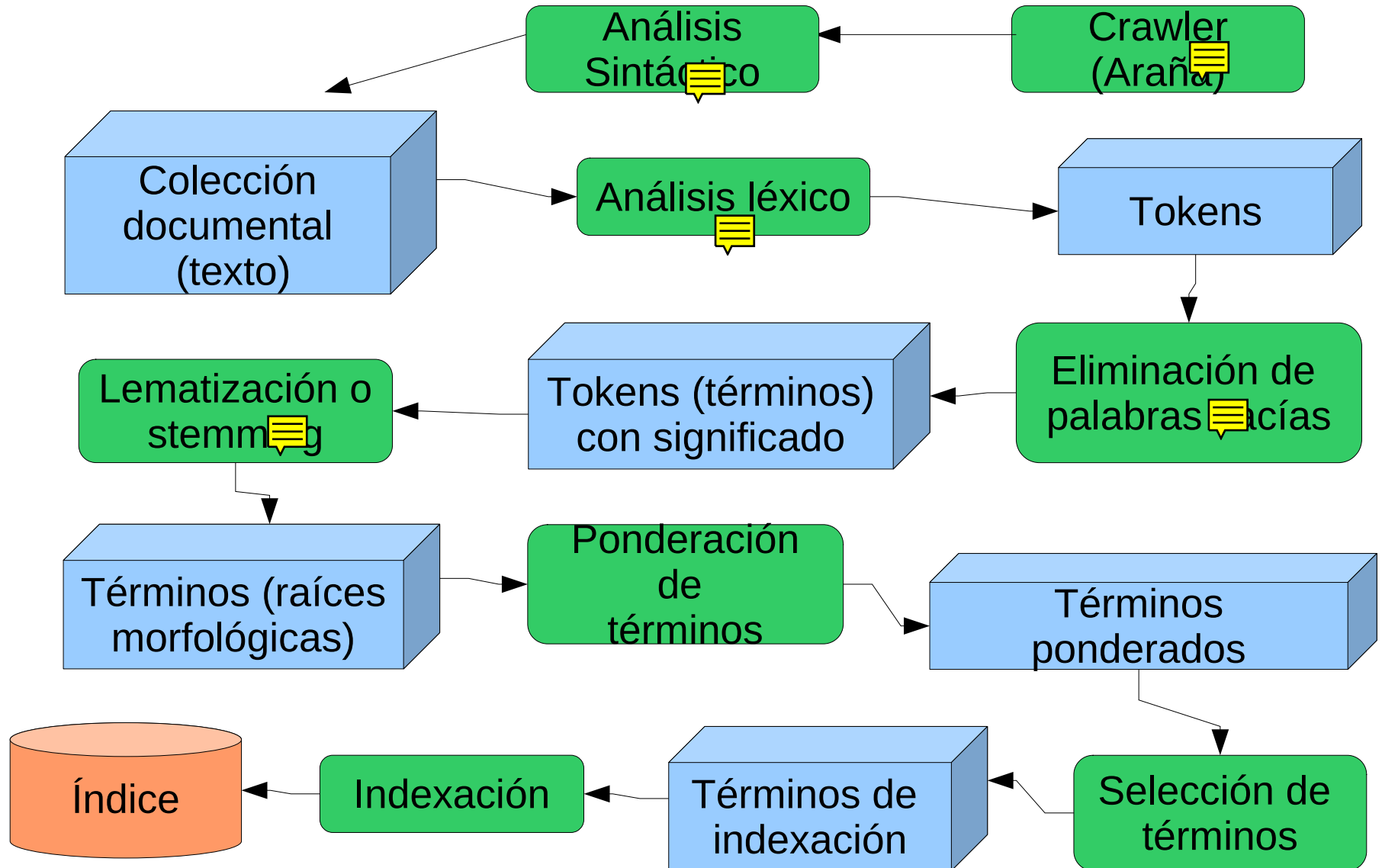
- El objetivo del procesamiento de texto es permitir una **representación**, lo más **eficiente** y **precisa** posible de un documento para facilitar su procesamiento por una máquina
- Hay que analizar estos documentos primero para extraer el contenido textual del mismo.



Motivación

- El lenguaje es la forma natural de comunicación del ser humano, y la escritura ha sido la herramienta fundamental para transmitir ideas, pensamientos y conocimientos..
- Gracias a la digitalización, la cantidad de información disponible ha crecido exponencialmente, abarcando desde documentos electrónicos y libros digitales hasta artículos científicos y publicaciones en línea.
- Usualmente ... las colecciones documentales están en un formato (PDF, Word, HTML, etc.) que no puede ser tratado directamente por una computadora:
 - **Contenido textual,**
 - **Contenido estructural:**
 - Elementos que dan formato al texto (negritas, cursivas, colores, etc.)
 - Elementos en tablas, imágenes
 - Enlaces para facilitar la navegación
 - Metadatos, como pueden ser la fecha de creación, autor, las palabras clave o las revisiones del documento.

El pre-procesamiento de texto



Paso 1. Analizador sintáctico (parser)

- Las colecciones documentales suelen estar en un formato que no puede ser tratado directamente por un Sistema de Recuperación de Información (SRI) ni por herramientas de Procesamiento de Lenguaje Natural (PLN),
 - Ejemplos de formatos originales:
 - Texto sin procesar (PDF, Word, HTML, etc.).

```
File View
</head>
<body><p/>
<div><p/>
<p><b>Introducción a la Recuperación de Información</b></p>
</div>
<div><p/>
</div>
<div><p/>
</div>
<p class="notes"><div/>
</p><div><p>En la asignatura ....</p>
</div>
<div><ul> <li><p>estudiaremos los fundamentos para poder construir herramientas que
ayuden a:</p>
<p/>
<li><p>Recuperación de Información (Text Retrieval, Search Engines).</p>
<li><p>Permitir a los usuarios encontrar la información que necesitan en cada
momento.</p>
<p/>
</li>
</ul>
```

```
Apache Tika: Unidad1-IntroduccionRI.pdf
File View
DefaultParser"/>
<meta name="X-TIKA:Parsed-By" content="org.apache.tika.parser.pdf.
PDFParser"/>
<meta name="access_permission:can_modify" content="true"/>
<meta name="pdf:docinfo:producer" content="LibreOffice 24.2"/>
<meta name="pdf:docinfo:created" content="2024-09-18T08:47:32Z"/>
<title>Sistemas de Recomendación</title>
</head>
<body><div class="page"><p/>
<p>Introducción a la
Recuperación de
</p>
<p>Información</p>
<p/>
</div>
<div class="page"><p/>
<p>
</p>
<p>En la asignatura ....
● estudiaremos los fundamentos para poder construir
</p>
```

Introducción a la Recuperación de Información

En la asignatura ...]

ayuden a: estudiaremos los fundamentos para poder construir herramientas que

Recuperación de Información (Text Retrieval, Search Engines).
Permitir a los usuarios encontrar la información que necesitan en cada

Paso 1: El Analizador Sintáctico (Parser)

Extracción de Contenido

```
<html><body><h1>
```

Bienvenido

```
</h1><p>
```

Este es un

** ejemplo**

```
</b></p></body>
```

```
</html>
```

Eliminación de etiquetas estructurales para retener texto limpio.

Gestión de Codificaciones



ISO-8859-1

→ b'introducci\xf3n'

UTF-8

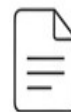
→ b'introducci\xc3\xbf3n'

El problema de la igualdad: el mismo carácter genera realidades binarias distintas.

Detección de Idioma



Reglas Heurísticas:
frec('the') > umbral → Inglés

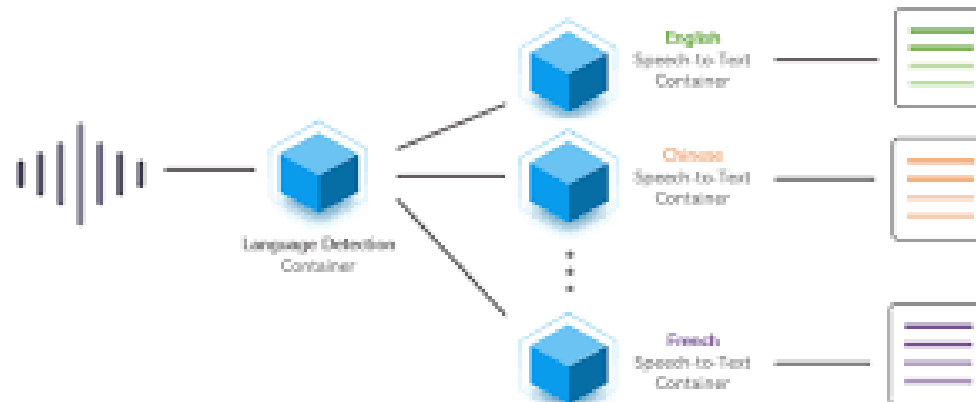


Clasificador Estadístico
(Machine Learning)

Enruta el texto al pipeline morfológico adecuado (ej. diferenciar lobo/lobos vs wolf/wolves).

Paso 1. Analizador sintáctico (parser)

- Cada una de estas cuestiones (detección formato, codificación, idioma) se puede obtener de los metadatos del fichero
- Pero en muchos casos representa un problema de clasificación (que veremos después en el curso)
 - se puede hacer de forma heurística,
 - mediante reglas ... si hay muchos 'the' => Inglés
 - o utilizando aprendizaje automático



Paso 2. Análisis léxico (tokenization)

Objetivo: Transformar la información textual en un conjunto de bloques (**tokens**) estructurados y procesados para permitir su análisis, indexación y búsqueda.

Pero... ¿qué es un token?



- Cualquier unidad básica de texto extraída de un documento
- Los tokens son agrupaciones de caracteres con un significado colectivo.

Paso 2. Análisis Léxico

- ¿Cómo identificar los tokens?
 - Manipulación de String,
 - Concatenar, dividir, búsqueda de subcadenas, comparar
 - Utilizar expresiones regulares
 - Podemos detectar ocurrencias de números, direcciones de email, errores de tipografía como “El el” en texto, etc
 - `Java.util.regex`
 - p.e. `[^.!?¿¡]+[.!?]` captura hasta punto, exclamacion o interrogación
 - Mastering regular expresion (Jeffrey Field)

Paso 2. Análisis Léxico

- El primer paso es dividir el string en pequeños bloques, tokens .. términos.
- Problema: ¿Qué será un token?
 - Dividir el string por espacios en blanco
 - `String[] salida = cadena.split("\\s+");`

Luís come patatas en el campo

Luís | come | patatas | en | el | campo

Paso 2. Análisis Léxico

- El primer paso es dividir el string en pequeños bloques, tokens .. términos.
- Problema: Qué es un término?
 - Dividir el string por espacios en blanco
 - `String[] salida = input.split("\\s+");`

Ella dijo: “No me puedo creer que el Madrid ganará la copa de S.M. el Rey en 2015-16”.

Ella | dijo: | “No | me | puedo | creer | que | el |
Madrid | ganará | la | copa | de | S.M. | el | Rey | en
| 2015-16”. |

- Usar reglas simples no funciona en la mayoría de los casos.

Paso 2. Análisis Léxico

- El primer paso es dividir el string en pequeños bloques, tokens .. términos.
- Problema: ¿Qué es un término?
 - Podemos utilizar el estándar definido por Unicode **UAX #29** para dividir texto en unidades significativas
Lucene (StandardTokenizer) lo usa

Ella dijo: “No me puedo creer que el Madrid ganará la copa de S.M. el Rey en 2015-16”.

Ella | dijo | No | me | puedo | creer | que | el |
Madrid | ganará | la | copa | de | S.M | el | Rey |
en | 2015 | 16 |

Paso 2. Análisis léxico (tokenization)

Los tokens son agrupaciones de caracteres con un significado colectivo.

- Problemática de la Granularidad: la elección de qué es un **token** depende de la aplicación concreta que estemos considerando.
- ¿Qué elementos son útiles para nuestra tarea?

Una misma cadena se puede procesar a diferentes escalas

- Frases, Palabras, caracteres, n-gramas (secuencia de palabras de longitud definida), partes de palabras.

Paso 2. Tipos de tokenización

RI → (términos de indexación). PLN → atributos

- Char "hola" → [h, o, l, a]: Algunos idiomas sin espacios claros, manejo de errores tipográficos
- Partes de Palabras "desconectado" → [des, conect, ado]: Hay mucho vocabulario y palabras raras (out-of-vocabulary). Traducción automática, LLMs,
- Palabras "El gato duerme" → [El, gato, duerme]: Lenguas con espacio claro (español, inglés). Recuperación de Información y Machine Learning clásico
- Ngramas "El gato duerme" → [El_gato, gato_duerme]: Capturar contexto local (reconocimiento de entidades), modelos de lenguaje clásicos, mejora RI
- Sentencias "El gato duerme. La casa esta tranquila" → [El_gato_duerme, La_casa_esta_tranquila]: La tarea opera a nivel de unidad semántica mayor (resumen, traducción, extracción de información).

Paso 2. Análisis léxico

- No hemos terminado !!!
- ¿Qué hacemos con...?
 - Números:
 - Normalmente ignorarlos.
 - Pero... podemos encontrarnos con fechas (2006, 450 A.C.,...) o con otros que representen informaciones relevantes al dominio.
 - Mezclados con letras pueden tener sentido (por ejemplo, CC123, B52, H2O).
 - Normalización: Mayúsculas y minúsculas:
 - Se pasan a minúsculas.
 - Hashtag, emoji, emails, URLs
 - En algunas aplicaciones puede ser interesante mantenerlos

Paso 2. Análisis léxico

- ¿Qué hacemos con...?

Guiones:

- Tratados como separadores: *state-of-art* -> *state of art*.
- Se ignoran: *on-line* → *online*.
- Mantenerlos: Algoritmo de *Knuth-Morris-Pratt*.

Problema con entidades: suelen ser nombres propios, conceptos, lugares, etc.

- Si obtenemos 2 tokens, (dividimos los guiones) entonces las consultas con 1 sólo termino emparejarán
 - “Albert-Einstein” -> [Albert, Einstein]
 - q = {Einstein} OK q = {Albert} ???
- Si consideramos un sólo token, entonces las consultas relativas a un termino no emparejan
 - “Albert-Einstein” -> [Albert-Einstein]
 - q = Einstein NO

Palabras raras o fuera del vocabulario (OOV)?

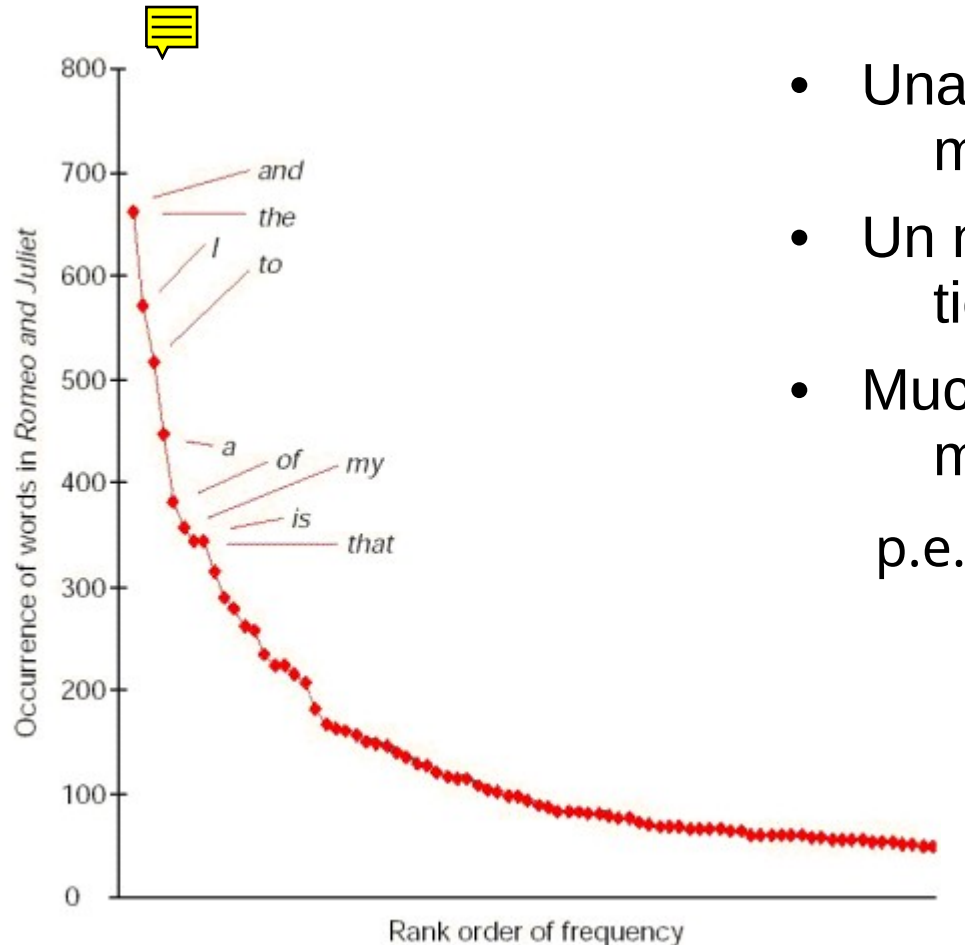
- La solución moderna (usada por LLMs) es utilizar subpalabras,
(ver BPE al final de estas transparencias)



Paso 3. Eliminación de palabras vacías

- Eliminar del diccionario aquellas palabras más comunes (alta frecuencia)
 - Tienen poca semántica (artículos, determinantes, pronombres, preposiciones,...)
 - Son palabras muy frecuentes (más del 80%) y poco útiles para la recuperación: ahorramos espacio y ganamos eficacia recuperadora.
- Aunque depende de la aplicación, la tendencia hoy día es no hacerlo.
 - Se indexan y se determina su uso en el proceso de búsqueda

Paso 3. Eliminación de palabras vacías



- Unas pocas palabras ocurren muy frecuentemente.
- Un número medio de palabras tienen una frecuencia media.
- Muchas palabras ocurren con muy poca frecuencia.

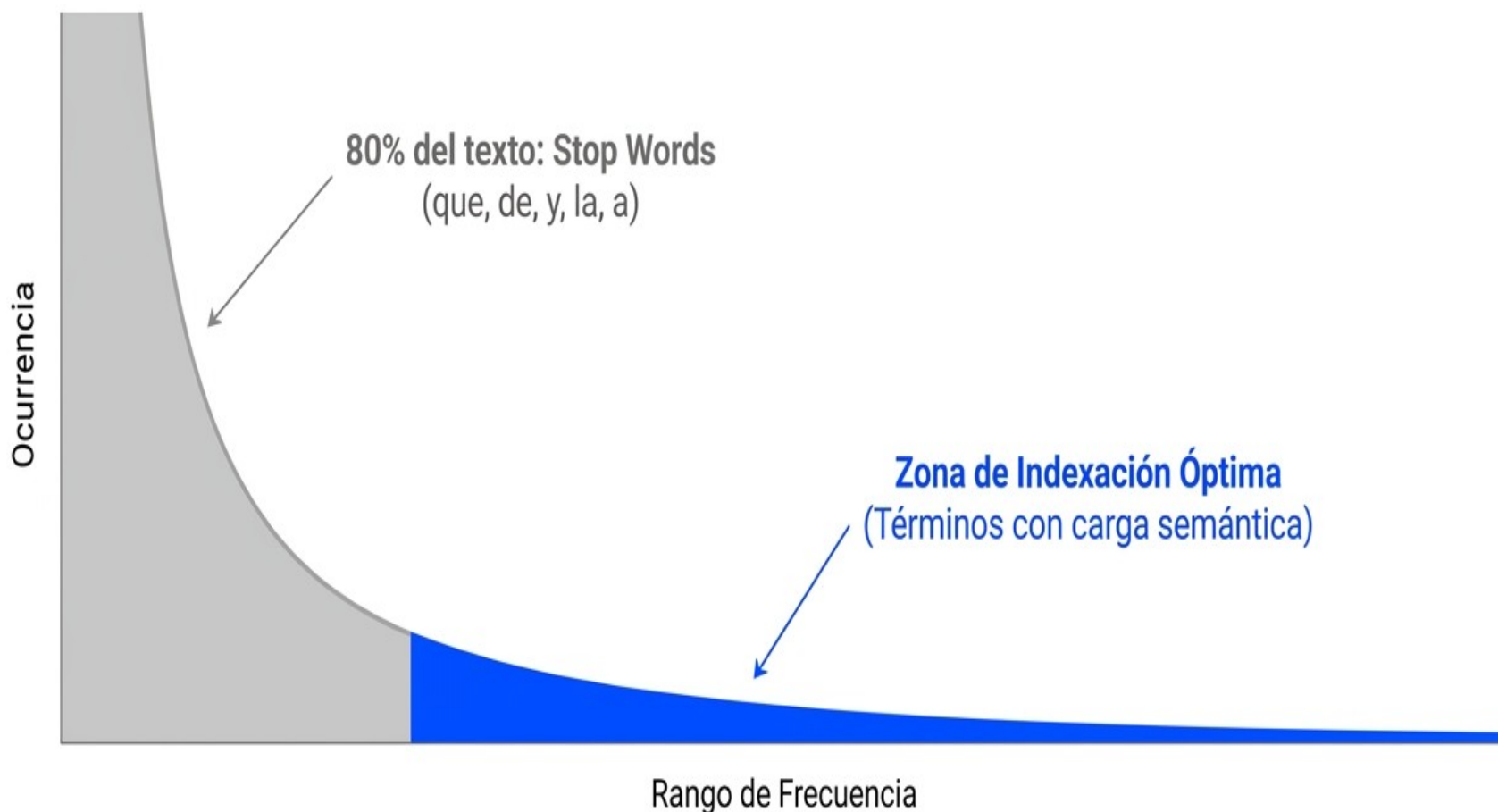
p.e. Quijote:

- Las 5 palabras más frecuentes en el Quijote son: “que, de, y, la, a”, representando el 20% del total.
- Un 10% de ellas ocurren 1 o 2 veces.

La Justificación Matemática: Ley de Zipf

La distribución del vocabulario humano no es aleatoria, es altamente sesgada. Filtrar la cabeza de la curva ahorra espacio masivamente sin sacrificar la capacidad recuperadora.

$$F(r) * r^s = k$$



Paso 3. Eliminación de palabras vacías

Palabra	Frecuenc.	Posición	f * r
the	3332	1	3332
and	2972	2	5944
a	1775	3	5235
he	877	10	8770
be	294	30	8820
there	222	40	8880
one	172	50	8600
friends	10	800	8000

Paso 3. Eliminación de palabras vacías

- Dependen del idioma.
- Y del dominio:
 - Podemos incluir palabras muy comunes que aportan poco en colecciones concretas: “ordenador” en una colección de documentos sobre informática).

Paso 3. Eliminación de palabras vacías

En español:

un, una, unas, unos, uno, sobre, todo, también, tras, otro, algún, alguno, alguna, algunos, algunas, ser, es, soy, eres, somos, sois, estoy, esta, estamos, estáis, estan, como, en, para, atrás, porque, por qué, estado, estaba, ante, antes, siendo, ambos, pero, por, poder puede, puedo, podemos, podéis, pueden fui, fue, fuimos, fueron, ...

En inglés:

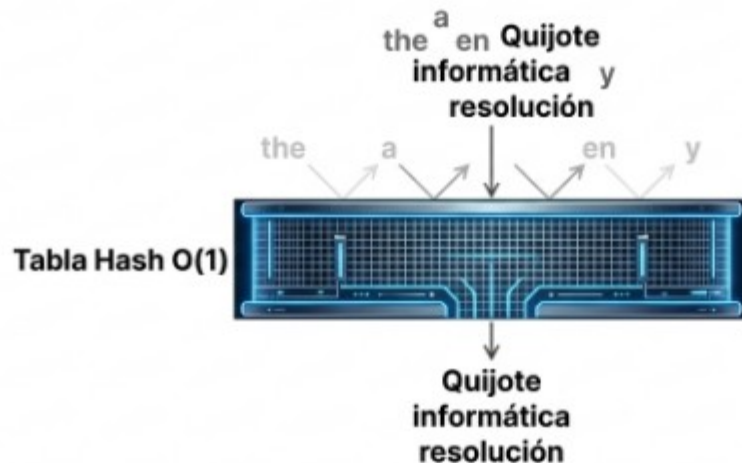
a, about, above, across, after, again, against, all, almost, alone, along, already, also, although, always, among, an, and, another, any, anybody, anyone, anything, anywhere, are, area, areas, around, as, ask, asked, asking, asks, at, away, b, back, backed, backing, backs, be, because, became, ...

Paso 3. Eliminación de palabras vacías

- Consulta: “¿Ser o no ser?” , “the who” :-)
- Tendencia:
 - Lista de palabras vacía pequeña para colecciones generales o usuarios poco experimentados.
 - Más completa para dominios muy definidos y los usuarios están entrenados en el uso de esas colecciones.

Paso 3. Eliminación de palabras vacías

- Implementación:
 - Inclusión de la lista de palabras vacías en una estructura de datos de acceso rápido (**tablas Hash**, por ejemplo).
 - Para cada palabra del texto, comprobar si está en dicha estructura. Si lo está, se descarta. En caso contrario, considerarla para la siguiente etapa.



Paso 4. Normalización lingüística

Normalizar las palabras es la tarea de poner los tokens en un formato estándar.

- Convertir a minúsculas es la forma más simple de normalización.
 - [Lucene] y [lucene]
 - [España] y [españa]

pasan a ser el mismo concepto, útil para la generalización. Una consulta emparejaría con documentos que contienen ambos términos

- [Granada] y [granada] ??



Pero existen otras estrategias para normalizar

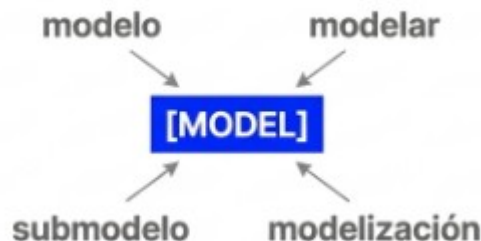
Paso 4. Normalización lingüística

Métodos usados para reducir el tamaño del vocabulario.



‘El **modelo** de lenguaje basado en redes neuronales permite **modelar** textos complejos con mayor precisión. Los **modelos** entrenados con grandes volúmenes de datos ... es un **submodelo** de ... una **modelización** ... está **modelado** ... **modelará** ... y **modelados** según el estilo deseado.’

La palabra modelo y sus variantes aparece una única vez, sin embargo todas hacen referencia la mismo concepto abstracto.



Paso 4. Normalización lingüística

Métodos usados para reducir la dimensionalidad del vocabulario.

- Es un proceso de normalización lingüística en el cual las diferentes formas que puede adoptar una palabra son reducidos a una única forma común, a la cual se denomina raíz, **stem o lema**.



- En lugar de indexar todas las palabras, se buscan sus “representantes” morfológicos, raíces. La raíz es la parte de la palabra que queda al eliminar afijos (prefijos y sufijos).

Paso 4. Normalización lingüística

- Ventajas:
 - Reducción del vocabulario → eficiencia y ahorro de espacio.
 - Aumenta el número de documentos recuperados.
- Desventajas:
 - Se pierde información sobre la palabra completa.



Es crítico para la calidad de la recuperación, tenemos que tener un compromiso entre **recall vs precision**

Poca/Nula Normalización

Fuerte Normalización



Precisión

Sube Precisión, Cae Exhaustividad (Recall).
Solo encuentra coincidencias exactas.

Exhaustividad (Recall)

Sube Exhaustividad (Recall), Cae Precisión.
Recupera más documentos, pero introduce ruido léxico.

Paso 4.1. Segmentación (Stemming)

- El stemming es el proceso de reducir una palabra a su raíz o forma base, eliminando sufijos y prefijos.
- Utiliza reglas heurísticas y algoritmos simples para cortar los extremos de las palabras. No necesariamente produce una palabra válida o real en el idioma.
- El análisis morfológico necesario para extraer la raíz es dependiente del idioma y puede llegar a ser muy complejo (como en el español).
- Lo habitual es que los métodos automáticos de segmentación eliminen los afijos de manera iterativa y “ciega”.
- Puede ser menos preciso, ya que a veces produce formas₃₃ que no son palabras reconocibles en el idioma.

Paso 4.1. Stemming

Ejemplos

biblioteca, bibliotecas, bibliotecario,
bibliotecarios, bibliotecaria, bibliotecarias,

Se podrían aglutinar en **bibliotec**



Supongamos que tenemos la frase:

"Los niños estaban jugando en el parque."

Al aplicar el stemming, podríamos obtener:

"Los niñ estab jug en el parqu."

3.1. Stemming

- Ejemplos en inglés:
 - {*connected*, *connecting* *connection*, *connections*, *disconnected*} → **connect**
 - {*computer*, *computational*, *computation*} → **comput**

for example compressed
and compression are
both
accepted as equivalent to
compress.



exampl compress
compress
accept equival compress.

Taxonomía de los Algoritmos de Stemming Automáticos

- Uso de **tablas**:
 - Los términos y sus correspondientes lexemas pueden estar almacenados en una tabla realizando la lematización a través de búsquedas en esa tabla.
- Algoritmos de eliminación de afijos
 - eliminan sufijos y/o prefijos del término y deja un lexema.

4.1.1. Eliminación de afijos

- Este tipo de algoritmos están basados en métodos heurísticos.
- Aplican sucesivamente reglas a las palabras.
- Palabras diferentes pueden dar lugar a una misma raíz léxica (lexema).
 - Algoritmos comunes en inglés:
 - Lovins, Slaton, Dawson, **Porter**, **Krovetz**, **snowball**

3.1.1. Eliminación de afijos

- Algoritmo de Porter:
 - Originalmente diseñado para inglés.
 - Utiliza una lista de sufijos y prefijos para eliminarlos.
 - Aplica una serie de reglas a los afijos de las palabras (unas 30 o 40 reglas).
 - Puede dar lugar a raíces que no son reconocidas como raíces en el idioma.
 - Ambigüedad en ciertos casos y errores en otros.

Porter algorithm

(Porter, M.F., 1980, An algorithm for suffix stripping,
Program, **14**(3) :130-137)

- Step 1: plurals and past participles
 - SSES -> SS caresses -> caress
 - (*v*) ING -> motoring -> motor
- Step 2: adj->n, n->v, n->adj, ...
 - (m>0) OUSNESS -> OUS callousness -> callous
 - (m>0) ATIONAL -> ATE relational -> relate
- Step 3:
 - (m>0) ICATE -> IC triplicate -> triplic
- Step 4:
 - (m>1) AL -> revival -> reviv
 - (m>1) ANCE -> allowance -> allow
- Step 5:
 - (m>1) E -> probate -> probat
 - (m > 1 and *d and *L) -> single letter controll -> control

Cómo hacerlo? Snowball

- Snowball es un lenguaje que permite representar reglas de stemming de forma sencilla
- Snowball permite traducir un script de snowball (.sbl) a un programa en ANSI C o Java

<http://snowball.tartarus.org/>

Demos:

<http://snowballstem.org/demo.html>

<http://text-processing.com/demo/stem/> 40

<http://snowball.tartarus.org/algorithms/spanish>

define standard_suffix as

```
( [substring] among(
    'anza' 'anzas'
    'ico' 'ica' 'icos' 'icas'
    'ismo' 'ismos'
    'able' 'ables' 'ible' 'ibles'
    'ista' 'istas'
    'oso' 'osa' 'osos' 'osas'
    'amiento' 'amientos'
    'imiento' 'imientos'
    (
        R2 delete
    )
)
```



```
'adora' 'ador' 'aci{o}n'
'adoras' 'adores' 'acciones'
'ante' 'antes' 'ancia' 'ancias'
(
    R2 delete
)
```

Ejemplos:

temer – oso; **temer** --osas

comput – adora; **comput** – acciones

Ejemplo: snowball

- En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un hidalgo de los de lanza en astillero, adarga antigua, rocín flaco y galgo corredor.
- en un lug de la mancha, de cuy nombr no quier acordarme, no ha much tiemp que viv un hidalg de los de lanz en astillero, adarg antigua, rocin flac y galg corredor.

Paso 4.2. Lematización



- Transformación de la palabra su forma canónica o lema, que es la forma básica que encontramos en un diccionario, considerando su significado y categoría gramatical.
- Requiere de un análisis morfológico más profundo y de conocimiento sobre el significado de las palabras. Busca la forma correcta que corresponde a una categoría gramatical.
- Por ejemplo:
 - Formas verbales a infinitivo:
 - {Fui, van, iremos} → ir
 - Femeninos a masculinos:
 - {gata} → gato
 - Casa, casas, casita, ... -> casa.

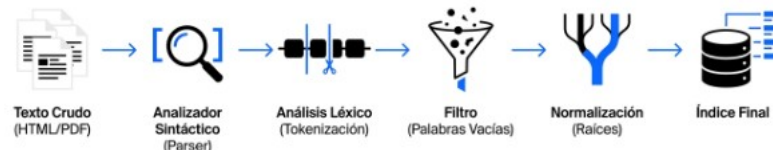
3.2. Lematización

- *Ejemplo:*
 - *the boy's cars are different colors → the boy car be different color*
- Más precisa que la segmentación.
- Produce palabras válidas en el idioma
- Necesita de más recursos (diccionarios de palabras).
- Necesita de las disciplinas de “Procesamiento del lenguaje natural” y “Lingüística computacional”.

Normalización Lingüística: Stemming vs. Lematización

Dimensión	Stemming (Algoritmos de Porter / Snowball)	Lematización
Base	Heurística basada en reglas de truncamiento matemático	Diccionarios morfológicos y análisis gramatical contextual
Velocidad	Extremadamente rápida (ideal para índices masivos)	Computacionalmente costosa
Resultado	Raíz truncada, a menudo no es una palabra real (ej. corriendo -> corr)	Lema morfológicamente correcto (ej. corriendo -> correr)
Precisión	Baja (riesgo de over-stemming/under-stemming)	Alta (resuelve ambigüedades morfológicas complejas)

Síntesis: El Viaje Completo



En resolución, don Quijote se enfrascó tanto en su lectura, que se le.... claro, y los días de turbio...

1. Tokenización (Unicode UAX #29)

[En] [resolución] [don] [Quijote] [se] [enfrascó] [tanto] [en] [su] [lectura,]
[que] [que] [se] [le...] [claro,] [y] [los] [días] [de] [turbio...]

2. Filtro Stop Words (en, se, su, los)

[resolución] [don] [Quijote] [enfrascó] [lectura] [claro] [días] [turbio]

3. Normalización Lingüística (Stemming)

[resoluc] [don] [quijote] [enfrasc] [lectura] [claro] [dia] [turbi]

Hemos convertido la literatura humana en un vector estructurado,
matemáticamente listo para indexación.

A nivel práctico

En **Lucene** (y sus derivados), utilizar un analizador

Los analizadores permiten la normalización de texto, la eliminación de palabras irrelevantes (stop words) y otros procesos de preprocesamiento. Son combinación de un tokenizer (tokenizador) y filters (filtros).

- StandardAnalyzer: Es el analizador más común. Realiza la tokenización y la normalización, eliminando palabras vacías (inglés) y aplicando minúsculas.
- WhitespaceAnalyzer: divide el texto solo en función de los espacios en blanco. No realiza ninguna normalización adicional.
- SpanishAnalyzer: diseñado para textos en español y utiliza un conjunto de stop words en español y un tokenizador apropiado.
- Otros

Herramientas Prácticas I: Ecosistema Apache Lucene

Un Analyzer en Lucene es un pipeline encapsulado (Tokenizer + Filters) listo para producción.

Input Crudo: 'Los gatos, en el parque.'

WhitespaceAnalyzer

Divide solo por espacios.
Cero normalización. Retiene máxima cantidad de ruido.

[Los] [gatos,] [en]
[el] [parque.]

StandardAnalyzer

Aplica UAX #29, normaliza a minúsculas, elimina stop words (lista en inglés).

[los] [gatos] [en]
[el] [parque]

SpanishAnalyzer

Pipeline adaptado:
Tokenizador + Stop words ES + Stemmer ES.

[gat] [parqu]

```
import org.apache.lucene.analysis.es.SpanishAnalyzer;
import org.apache.lucene.analysis.TokenStream;
import org.apache.lucene.analysis.tokenattributes.CharTermAttribute;
import java.io.StringReader;

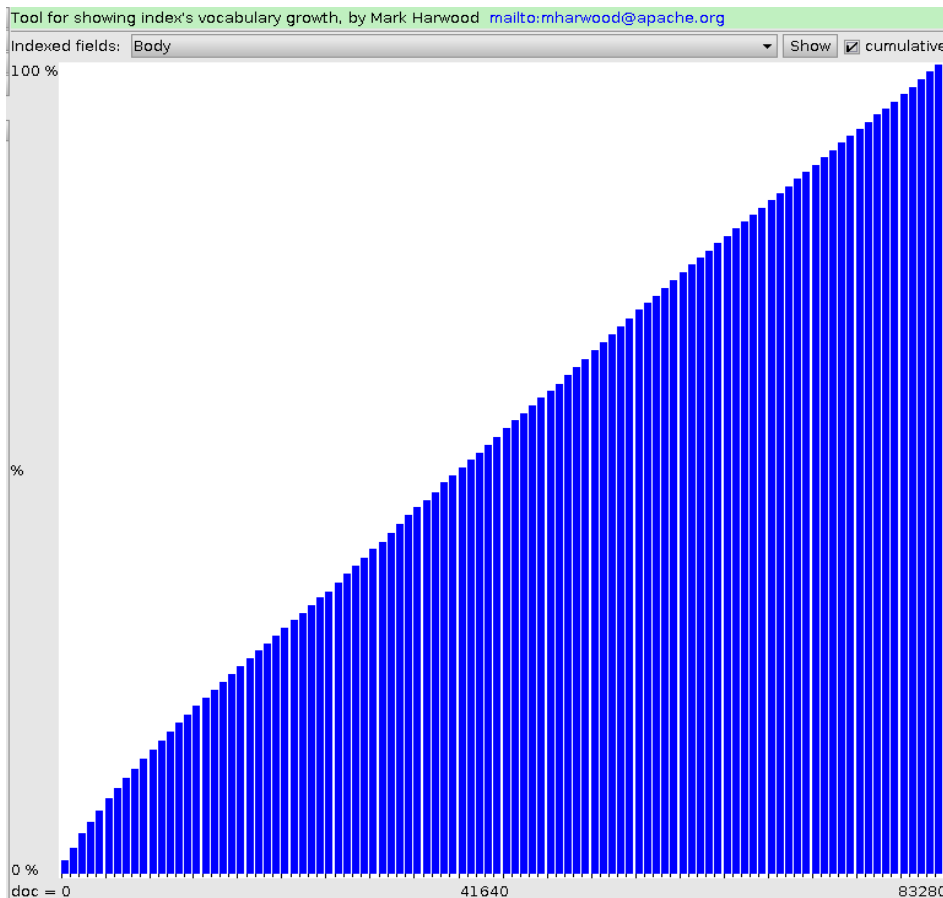
public class SpanishAnalyzerExample {
    public static void main(String[] args) throws Exception {
        String text = "Los gatos están jugando en el parque.";
        SpanishAnalyzer analyzer = new SpanishAnalyzer();

        try (TokenStream tokenStream = analyzer.tokenStream("field", new StringReader(text))) {
            CharTermAttribute charTermAttribute = tokenStream.addAttribute(CharTermAttribute.class);
            tokenStream.reset();
            while (tokenStream.incrementToken()) {
                System.out.println(charTermAttribute.toString());
            }
            tokenStream.end();
        }
    }
}
```



Ejemplo de analizadores

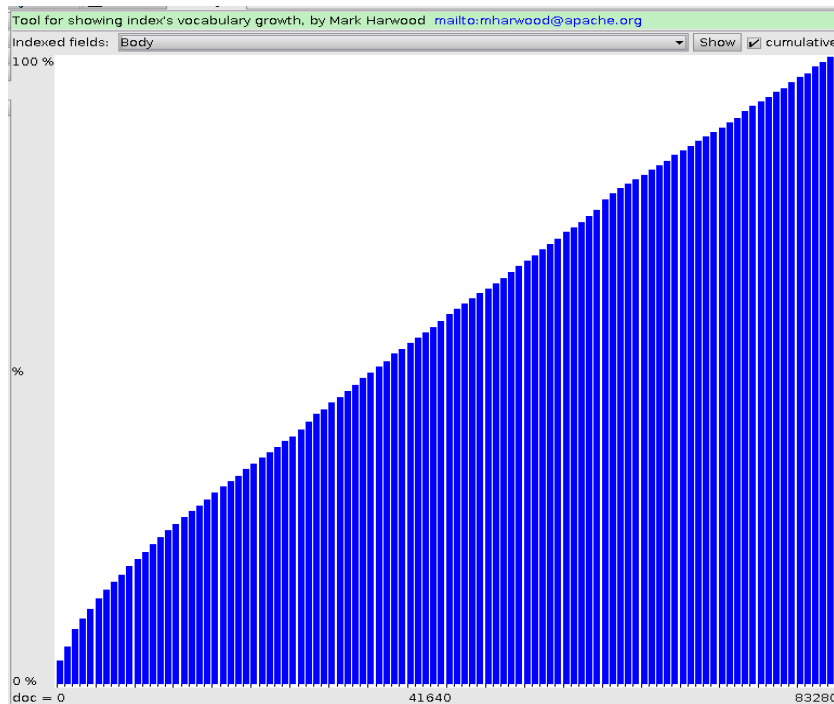
- WhiteSpaceAnalyzer
 - #docs 83280
 - #terms 998623



Top ranking terms. (Right-click for more op

Rank	Freq	Field	Text
1	83280	Body	"
2	77841	Body	the
3	70491	Body	of
4	69782	Body	to
5	69240	Body	is
6	67953	Body	a
7	64674	Body	and
8	58203	Body	in
9	56806	Body	you
10	55873	Body	that
11	53162	Body	for
12	48797	Body	be
13	48379	Body	are
14	43499	Body	with
15	40660	Body	can
16	40546	Body	it
17	39973	Body	as
18	39799	Body	this
19	38523	Body	not
20	38208	Body	on
21	36943	Body	have
22	35530	Body	your
23	32952	Body	</p>
24	31387	Body	I
25	31040	Body	by
26	30810	Body	if
27	30615	Body	but
28	30337	Body	or
29	29979	Body	an
30	28740	Body	from

- StandardAnalyzer
 - divide por no-letras
 - convierte a minúsculas
 - Quita palabras vacías.
 - #docs 83280 #terms 251448



action means all fields.

Top ranking terms. (Right-click for more option)

Rank	Freq	Field	Text
1	83083	Body	p
2	61026	Body	you
3	42854	Body	i
4	41231	Body	can
5	38029	Body	have
6	36821	Body	your
7	33885	Body	1
8	33148	Body	href
9	31881	Body	http
10	31002	Body	data
11	30691	Body	from
12	29810	Body	which
13	29735	Body	so
14	28542	Body	one
15	25925	Body	2
16	25760	Body	would
17	25466	Body	rel
18	25387	Body	nofollow
19	25099	Body	more
20	25025	Body	use
21	23065	Body	some
22	22981	Body	model
23	22292	Body	do
24	21875	Body	what
25	21779	Body	all
26	20132	Body	using
27	20119	Body	also
28	19855	Body	code
29	19469	Body	em
30	19249	Body	other

A nivel práctico

En **NLTK** (Natural Language Tool Kit), biblioteca Python para procesamiento de lenguaje natural

Tiene métodos como:

- `word_tokenize` para dividir texto en palabras
- `sent_tokenize` para dividir texto en sentencias
- Dispone de conjuntos de palabras vacías para distintos idiomas
- Contiene `PorterStemmer` y `SnowballStemmer` (entre otros) para la eliminación de sufijos y `WordNetLemmatizer` para obtener la forma canónica de las palabras

Herramientas Prácticas II: NLTK en Python

El Natural Language Toolkit (NLTK) es el estándar programático para investigación y Procesamiento de Lenguaje Natural.

```
import nltk
from nltk.stem import PorterStemmer

texto = "Los gatos están jugando"

# 1. Tokenización (UAX #29)
tokens = nltk.word_tokenize(texto)

# 2. Inicializar analizador morfológico
stemmer = PorterStemmer()

# 3. Aplicar segmentación ciega
stems = [stemmer.stem(token) for token in tokens]
```

Divide el texto aislando la puntuación de forma inteligente.

Aplica heurísticas rápidas de corte de sufijos sin validación de diccionario.

Tokenizando en partes de palabras

- Existe otro enfoque de tokenización utilizado por LLMs donde las palabras son descompuestas en trozos (usado en ChatGPT)
- La granularidad puede ir desde caracteres a palabras completas.
- **Byte Pair Encoding** (BPE) es uno de estos tokenizadores, que sigue la idea de fusionar caracteres frecuentes en un vocabulario.
- Ante una entrada como:

estamos comiendo patatas en el parque 📄

- Podríamos tener tokens, como:

estamos comien do patat as en el parque

o bien:

esta mos com iendo patatas en el par que

BPE

- Ventajas:
 - Balance entre palabras completas y caracteres.
 - Manejo eficiente de palabras raras o desconocidas.
 - Aprende subunidades útiles
- Desventajas:
 - La tokenización hay que aprenderla de un conjunto representativo de documentos

Usos:

- Tokenización en modelos de lenguaje.
- Compresión de texto.
- Base de algoritmos más avanzados

BPE. ¿Cómo funciona?

- Se comienza con un vocabulario de caracteres individuales.
- Se cuentan las frecuencias de pares adyacentes.
- El par más frecuente se fusiona en un nuevo símbolo.
- Se repite hasta alcanzar un número deseado de mezclas.

Ejemplo

Entrada: 'low lower lowest west'

Paso	Secuencia	Top pares (con frecuencias)
Paso 0	l o w _ l o w e r _ l o w e s t _ w e s t	lo:3, ow:3, we:3
Paso 1: mezcla (l,o)->lo	l o w _ l o w e r _ l o w e s t _ w e s t	low:3, we:3, _lo:2 ,...
Paso 2: mezcla (lo,w)->low	l o w _ l o w e r _ l o w e s t _ w e s t	lowe:2, es:2, st:2
Paso 3: mezcla (e,s)->es	l o w _ l o w e r _ l o w e s t _ w e s t	est:2 ...:1
Paso 4: mezcla (es,t)->est	l o w _ l o w e r _ l o w e s t _ w e s t	lowe:1, ...west:1
Paso 5: mezcla (s,t)->st	l o w _ l o w e r _ l o w e s t _ w e s t	

Así, aprendería low como raíz de palabra y est como sufijo

Vocabulario: _ l o w e r s t lo low es est west

Tokenización Moderna: Nivel de Palabra vs. Subpalabras (BPE)

Dimensión	Clásica (Nivel de Palabra)	BPE (Byte Pair Encoding)
Vocabulario	Fijo y masivo (cientos de miles de tokens)	Comprimido y eficiente (ej. 30k - 50k tokens)
Palabras Desconocidas (OOV)	Falla rotundamente (Asigna token de error [UNK])	Excelente. Divide la palabra desconocida en subpalabras conocidas.
Aplicación Principal	Motores de búsqueda tradicionales (Apache Lucene)	Large Language Models (LLMs - GPT, BERT)
Dependencia Lingüística	Requiere reglas específicas y mantenimiento por idioma	Algoritmo puramente estadístico (completamente agnóstico al idioma)